When Should a Kalman Filter Trust Its Forward Model? Closed-Loop Evidence from a Muscle-Driven Reaching Task

Jun Kobayashi

***Abstract*—We study how a learned forward model and a learned Kalman gain interact in a closed-loop muscle-driven reaching task. A residual MLP forward model and a fixed-lag Kalman filter blend predictions with delayed noisy observations of a 34-muscle MyoSuite arm; a small condition-level predictor maps (noise, delay, controller) to a scalar gain K trained against an open-loop oracle. Plugging the estimator into a joint-space PD controller with an inverse-kinematics target reveals a chain of findings. *First*, with a single-step-supervised forward model the learned predictor matches the oracle and beats observation-only ($K{=}1$) by 8.6 cm at the low-noise large-delay cell where blending matters most. *Second*, behaviour-cloned controllers reach the target more reliably but consume the estimator output weakly enough that the per-cell gap collapses to under 3 cm, suggesting that closed-loop estimator benchmarks need state-coupled controllers. *Third*, retraining the forward model with K-step rollout supervision ($K{\in}\{4, 8\}$) reduces $h{=}50$ tip-prediction error by 54–65% and shifts the closed-loop optimum to prediction-only ($K{=}0$), which now beats $K{=}1$ by 12–24 cm across the entire grid, while the condition-level predictor still outputs $K \approx 1$ because the open-loop oracle it was trained against never selects $K{=}0$. *Fourth*, redefining the predictor's training oracle in terms of closed-loop task error rather than open-loop estimation accuracy moves its output to $K{\approx}0.02$, matches the $K{=}0$ baseline within 1.5 cm at delay ≥ 18 steps, and closes 40–60% of the gap at zero delay — with no change to the predictor architecture. The right Kalman gain in closed loop therefore depends not only on the noise and delay regime but on *what objective the predictor was trained against*, and learned-Kalman-gain papers should specify the oracle choice explicitly.**

***Index Terms*—Forward model, Kalman filter, state estimation, multi-step rollout supervision, behaviour cloning, closed-loop control, MyoSuite, muscle-driven reaching.**

I. Introduction

State estimation in muscle-driven manipulation faces three coupled challenges: sensorimotor delay, observation noise, and signal-dependent motor noise. A common engineering response is the Kalman filter, which blends a forward model's prediction with the current observation via a gain $K \in [0, 1]$: $K{=}0$ trusts the model, $K{=}1$ trusts the sensor, and the literature on adaptive Kalman filters has produced many ways to choose K from the estimated noise covariance [1]–[3]. The internal-model view of biological motor control casts the same relationship as a forward-model / sensory-feedback combination [4], [5]. In this paper we ask a different question: *what does the optimal K look like in closed loop, and does the way we train a learned K-predictor match it?*

We attack the question on a 34-muscle MyoSuite reaching arm [6] running in the MuJoCo simulator [7]. We learn a residual-MLP forward model from rollouts, use it inside a fixed-lag Kalman filter that handles a configurable observation delay ($d{=}0$–36 steps) and configurable per-field Gaussian noise, sweep a $K{\in}\{0, 0.25, 0.5, 0.75, 1.0\}$ grid across noise and delay to define an oracle gain $K^\star$ per condition, and train a small condition-level predictor on those oracle labels. A separate joint-space PD controller with one-shot inverse-kinematics target then *deploys* the estimator in closed loop.

Our contributions are summarised in four claims.

- **C1.** Under a single-step-supervised forward model the oracle $K^\star$ falls from 1.0 to 0.25 across noise at zero delay but collapses uniformly to 1.0 at delay ≥ 18 steps, because the forward-model rollout error compounds faster than the observation noise grows (Fig. 2, left panel).
- **C2.** A learned condition-level predictor matches the oracle within 0.6 cm on average and beats $K{=}1$ by 8.6 cm in closed-loop reaching at the (low-noise, large-delay) cell where blending matters most (Fig. 3). Behaviour-cloned controllers improve reaching but wash out the estimator signal because the small demonstration set causes the policy to collapse to an open-loop average (Fig. 5); closed-loop estimator benchmarks require state-coupled controllers.
- **C3.** Retraining the forward model with K-step rollout supervision ($K{=}4$, then $K{=}8$) cuts $h{=}50$ tip-prediction error by 54–65% and widens the open-loop $K{<}1$ regime to delay ≥ 18 steps. In closed loop, however, prediction-only ($K{=}0$) becomes globally optimal: it beats $K{=}1$ by 12–24 cm in min-tip error, and the condition-level predictor still outputs $K{\approx}1$ because the open-loop oracle it was trained against never selects $K{=}0$ (Fig. 6, panels 1–3).
- **C4.** Replacing the predictor's training oracle with per-cell closed-loop K-sweep labels and retraining the same MLP yields $K{\approx}0.02$, matches the $K{=}0$ baseline within 1.5 cm at all delay-18 cells, and closes 40–60% of the gap at delay-0 cells (Fig. 6, panel 4). The residual delay-0 gap is structural: the K-curve there jumps sharply from

Manuscript received [date]; revised [date]. J. Kobayashi is with [Affiliation TBD] (e-mail: kobayashi.jun184@m.kyutech.ac.jp). This work was supported in part by [funding source TBD]. Code and configs are released at [repository URL TBD].

$K{=}0$ to $K{=}0.25$, so the predictor's sigmoid asymptote loses $\sim$0.17 m.

Together, C1–C4 say that the optimal closed-loop Kalman gain depends on two coupled choices that the literature usually treats as independent: the forward model's accuracy over the controller's planning horizon, and the oracle the K-predictor was trained against. When the open-loop oracle and the closed-loop optimum disagree — as they do for multi-step-supervised forward models on this task — the two pull the predictor in opposite directions.

The rest of the paper is organised as follows. Section II positions our work against prior literature on adaptive Kalman filtering, forward models in motor control, learned dynamics, imitation learning, and muscle-driven simulation. Section III describes the task, the forward model and its multi-step variant, the fixed-lag Kalman estimator, the three controllers, and the evaluation pipeline. Section IV reports the four claims with the corresponding figures. Section V discusses the trade-offs and the implications for learned adaptive Kalman gains.

II. Related Work

Adaptive Kalman gains. The Kalman filter [1] and its smoothing counterpart [11] are the textbook framework for combining a state-space model with noisy observations under known noise covariances. When the covariances are unknown, the classical *adaptive Kalman filter* estimates them online from the innovation sequence [2]; the steady-state gain K then drops out of the covariance estimate. The state-space formulation we adopt — residual MLP for dynamics, fixed-lag buffered correction for delays — is standard textbook material [3]. The novelty of our setting is not in the filter equations but in the choice of K: instead of deriving K from a noise-covariance estimate, we train a small condition-level predictor that maps (controller, σ, delay) to a scalar K supervised against a swept oracle. The contribution we report is what happens when that oracle is open-loop versus closed-loop, and that distinction is largely absent from the adaptive Kalman literature.

Forward / internal models in motor control. The view that the central nervous system maintains an internal forward model of the limb to predict the sensory consequences of motor commands traces back to Bernstein's degrees-of-freedom analysis [4] and was given an explicit computational framing by Wolpert and Ghahramani [5], who identified forward and inverse models as complementary processes underlying sensorimotor estimation, prediction, and learning. Our residual MLP is a coarse instantiation of the forward model in that picture; the multi-step supervision we add is motivated by the fact that the loop that consumes the estimate operates over hundreds of integration steps and thus depends on long-horizon prediction quality.

Learned dynamics and multi-step supervision. Model-based reinforcement-learning systems have moved from one-step dynamics losses to multi-step rollout objectives because closed-loop planners need predictions that survive being unrolled. Dreamer [9] backpropagates through imagined trajectories in a latent world model, and MBPO [10] branches short model rollouts from real data, formalising "when to trust your model". We take the same lesson into Kalman filtering: a forward model that is accurate enough over the controller's planning horizon shifts where blending is helpful and where it is harmful.

Imitation learning for closed-loop reach. Behaviour cloning has been used for closed-loop control since ALVINN [12]; the standard fix for the resulting covariate shift is DAgger [13]. We deliberately *do not* apply DAgger in this work and report the resulting open-loop collapse as a methodological observation (Sec. IV-D): at the demonstration scale we test, BC controllers reach better than the joint-PD baseline but consume state estimates weakly enough that estimator differences vanish, which we read as a cautionary note for closed-loop estimator benchmarks built on BC controllers.

Muscle-driven simulation. Our task runs in MuJoCo [7] via the MyoSuite musculoskeletal benchmark [6], which provides a 34-muscle reaching arm and a contact-rich set of tasks suitable for sensorimotor estimation studies. MyoSuite is increasingly used as a testbed for reinforcement-learning controllers; here we use it instead to expose the trade-off between forward-model accuracy and the training oracle of a learned Kalman gain, a question that the RL-centric literature has not addressed.

III. Method

A. Task and Environment

We use the `myoArmReachFixed-v0` task in MyoSuite 2.12.2, a 34-muscle shoulder-elbow-wrist-hand system actuated by activations in $[0,1]^{34}$. The task is to drive the index-finger tip to a randomly drawn target in a 12 s episode at $\Delta t{=}0.02$ s (600 control steps). The flat state $x \in \mathbb{R}^{83}$ stacks q (20), $\dot{q}$ (20), activation (34), tip position (3), target (3), and reach error (3). True state is extracted from MuJoCo and used only as an oracle for evaluation; the closed loop never accesses it (Fig. 1).

B. Forward Model

A residual MLP $f_\theta(x_t, u_t)$ predicts the state delta $\Delta x_t{=}x_{t+1}{-}x_t$ from a flat 83-dim state and 34-dim excitation input. Architecture: two hidden layers of 256 units with ReLU and LayerNorm. The model is optimised with Adam [8]. In the single-step baseline, f_θ is trained on the standard one-step MSE loss

$$\mathcal{L}_1(\theta) = \mathbb{E}_{(x_t,u_t,x_{t+1})}\big[\|f_\theta(x_t,u_t) - \Delta x_t\|^2\big]. \tag{1}$$

Multi-step rollout supervision. The multi-step variant trains f_θ with a K-step rollout loss instead of the one-step MSE. For each contiguous trajectory chunk $(x_t, u_t, \ldots, x_{t+K})$ inside a single source episode, the model is unrolled free-running: $\hat{x}_{t+k} = \hat{x}_{t+k-1} + f_\theta(\hat{x}_{t+k-1}, u_{t+k-1})$, $\hat{x}_t = x_t$, and the loss averages MSE across the K prediction steps:

$$\mathcal{L}_K(\theta) = \mathbb{E}\Big[\tfrac{1}{K}\sum_{k=1}^{K}\|\hat{x}_{t+k} - x_{t+k}\|^2\Big]. \tag{2}$$

Multi-step rollout supervision has been used to stabilise long-horizon predictions in latent-imagination world models [9] and

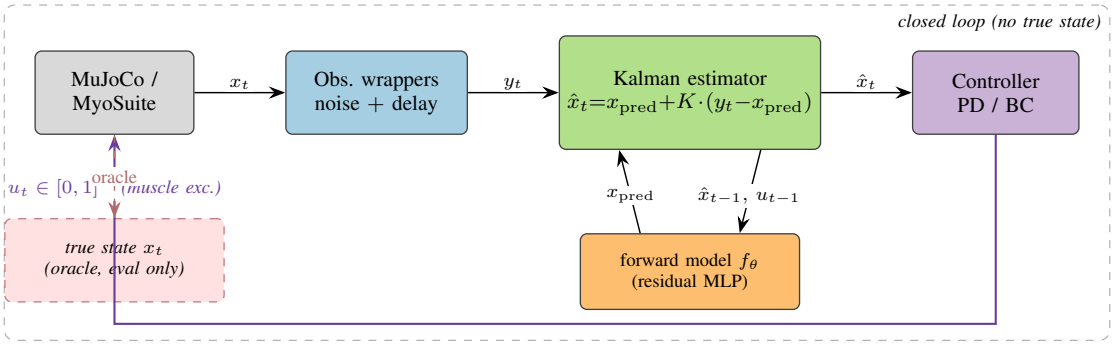

Fig. 1. **Closed-loop pipeline.** Each control step, the env's true state x_t is filtered through observation wrappers that add per-field Gaussian noise and a fixed delay, producing y_t. The Kalman estimator combines y_t with a one-step prediction x_{pred} produced by the residual-MLP forward model $f_\theta(\hat{x}_{t-1}, u_{t-1})$ via a scalar gain $K \in [0,1]$ (Eq. 4). The controller consumes only the estimator output $\hat{x}_t$ and emits the next muscle excitation $u_t \in [0,1]^{34}$. The true state x_t is exposed only to the offline evaluator (dashed red box); the closed loop never accesses it.

in model-based policy optimisation [10], where the relevant horizon is the planner's rollout depth rather than the estimator's delay; we re-use the technique here to make f_θ robust across the Kalman buffer.

We train the same architecture with $K \in \{1, 4, 8\}$ on the same expanded dataset (51 k transitions across 106 episodes; see Sec. IV-A).

C. Fixed-Lag Kalman Estimator

The observation pipeline produces $y_t = h(x_{t-d}) + \eta_t$ where h is the identity (the state vector and the observation share the same schema), $d \in \mathbb{Z}_{\geq 0}$ is a fixed delay in control steps, and η_t is per-field i.i.d. Gaussian noise with diagonal covariance $\text{diag}(\sigma^2)$. The estimator maintains a length-$(d{+}1)$ ring buffer of past one-step estimates $(\hat{x}_{t-d}, \ldots, \hat{x}_{t-1})$ together with the corresponding applied actions $(u_{t-d}, \ldots, u_{t-1})$. At each step the estimator first runs the forward model on every buffered estimate to obtain a prediction at the present time,

$$\tilde{x}_t = \hat{x}_{t-1} + f_\theta(\hat{x}_{t-1}, u_{t-1}), \tag{3}$$

then forms an innovation against the *delayed* buffer entry and applies an element-wise gain $K \in [0,1]^{83}$:

$$\hat{x}_{t-d} \leftarrow \hat{x}_{t-d} + K \odot \big(y_t - \hat{x}_{t-d}\big), \tag{4}$$

and finally re-rolls the corrected past estimate forward through the buffered actions to obtain the present-time estimate $\hat{x}_t$. Eq. (4) collapses to the standard one-step Kalman update when $d{=}0$. Throughout the paper K is a scalar broadcast across the 83 fields so the gain reduces to a single trust knob between forward prediction ($K{=}0$) and observation ($K{=}1$). This fixed-lag construction is the classical buffered Rauch–Tung–Striebel smoother specialised to a scalar gain [3], [11].

D. Learned Gain Predictor

A small MLP $g_\phi : \mathbb{R}^8 \to [0,1]$ predicts the scalar Kalman gain from a condition feature vector. The input

$$\xi = \big[\,\text{onehot}(c)\,;\ \sigma\,;\ d/d_{\max}\,\big] \in \mathbb{R}^8, \tag{5}$$

concatenates a 3-way controller one-hot $\text{onehot}(c)$, the 4-dim noise-sigma vector σ (`qpos`, `qvel`, `tip_pos`, `reach_err`), and a normalised delay $d/d_{\max}$ with $d_{\max}{=}36$ steps. The network is a $8 \to 32 \to 32 \to 1$ MLP with ReLU and a sigmoid output. The training oracle $K^\star$ is obtained by, for each of the 72 conditions in the stress grid, sweeping $K \in \{0, 0.25, 0.5, 0.75, 1.0\}$ and selecting the K that minimises a chosen criterion. The default criterion is the per-step open-loop estimation error $\|\hat{x}_t - x_t\|^2$ averaged over the episode; Sec. IV-F (C4) replaces this criterion with the closed-loop min-tip task error and retrains the same MLP g_ϕ unchanged.

E. Controllers

Three controllers are evaluated as closed-loop wrappers around the estimator.

(i) Heuristic projection. A fixed random projection $W \in \mathbb{R}^{34\times 3}$ (drawn once with a fixed seed from $\mathcal{N}(0,1)^{34\times 3}$) maps the estimated 3-dim reach-error $\hat{r}$ to a 34-dim muscle excitation through a sigmoid:

$$u = \sigma(b - g\, W\, \hat{r}), \quad b{=}0,\ g{=}5, \tag{6}$$

producing a state-sensitive baseline that does not actually reach.

(ii) Joint-space PD with inverse kinematics. For each target tip position $p^\star \in \mathbb{R}^3$ we solve $q^\star = \text{IK}(p^\star)$ once at episode start via a damped-Jacobian Newton iteration

$$q^{(k+1)} = q^{(k)} + J^\top \big(JJ^\top + \lambda^2 I\big)^{-1} \big(p^\star - p(q^{(k)})\big), \tag{7}$$

where $J = \partial p / \partial q$ is the tip-site position Jacobian obtained from MuJoCo's `mj_jacSite` and $\lambda^2{=}0.1$ is the damping. The arm is then driven by a joint-space PD law and a moment-arm muscle map:

$$u_{\text{joint}} = K_p\,(q^\star - \hat{q}) - K_d\,\hat{\dot{q}}, \tag{8}$$

$$u_{\text{muscle}} = \text{clip}(\alpha\,\text{ReLU}(M\,u_{\text{joint}}),\ 0,\ 1), \tag{9}$$

where $M \in \mathbb{R}^{34\times 20}$ is the muscle-tendon moment-arm matrix (the dense form of MuJoCo's sparse `actuator_moment` at the env's current pose), and $\alpha = 5$ is a scalar action scale. Tuned gains: $K_p{=}30$, $K_d{=}3$.

(iii) Behaviour-cloned MLP. A two-hidden-layer (256 unit) sigmoid policy is trained on 30 scripted IK demonstrations in

the spirit of [12]; we deliberately do not iterate the demonstrator with DAgger [13] and report the resulting open-loop collapse in Sec. IV-D.

F. Evaluation Pipeline

Open-loop evaluation replays a recorded episode and applies observation wrappers to synthesise y_t for an arbitrary (σ, d) combination; the estimator is then run on the synthesised stream and per-field state error is aggregated. *Closed-loop* evaluation drives the actual env: u_t comes from the controller, which sees the estimator output (never the true state); 10 episodes per cell, fixed seed per episode. The same evaluation grid (3 controllers $\times$ 6 noise levels $\times$ 4 delays $=$ 72 conditions for open-loop, focused $3{\times}2{\times}10{=}60$ for closed-loop) is reused across phases.

IV. Results

A. Forward-model improvement cycle

An initial forward model trained on the original $\sim$ 3.6 k transitions from 6 episodes had low single-step error but unbounded long-horizon rollouts ($h{=}50$ MSE $\sim$ 10.6). Collecting 100 additional `random` and `lowamp` episodes (a total of 51 k transitions across 106 episodes) and retraining the same architecture cut $h{=}50$ MSE to 0.052 — a $\sim 200\times$ improvement. We use this expanded dataset, and the resulting model, as the single-step baseline throughout the paper.

B. Where blending matters: open-loop oracle (C1)

Fig. 2 (left panel) plots the oracle $K^\star$ for the single-step-supervised forward model. At zero delay $K^\star$ decreases monotonically from 1.0 (no noise) to 0.25 (extreme noise), recovering the textbook intuition that noisy observations should be trusted less. At delay ≥ 18 steps, however, $K^\star$ is uniformly 1.0: forward-model error compounds faster than observation noise grows, so even noisy observations beat the propagated estimate. Only 18 of 72 conditions (25%) require $K^\star{<}1$.

C. Closed-loop differentiation, single-step model (C2)

We evaluate three estimators ($K{=}0$, $K{=}1$, Stage A learned) under the joint-PD controller on 3 noise $\times$ 2 delay $\times$ 10 episodes. Fig. 3 shows the min-tip differences between the learned predictor and the $K{=}1$ baseline. The learned predictor beats $K{=}1$ by 8.6 cm at (none, $d{=}18$) — exactly the regime where the open-loop oracle says blending is least useful, but where in closed loop the controller's reliance on the buffered prediction makes a tighter estimate pay off. Other cells differ by less than 2 cm.

The trajectory plot in Fig. 4 shows that at this cell $K{=}0$ diverges (the prediction-only rollout is too inaccurate over $d{=}18$ steps), $K{=}1$ approaches but oscillates around the buffered target, and the learned estimator reaches further.

D. Behaviour-cloned controllers wash out the signal (C2 cont.)

To check whether the 8.6 cm signal generalises, we replace the joint-PD controller with a small behaviour-cloned MLP trained on a 30-episode scripted IK demonstration set. Reaching improves substantially (success$<$0.1 m rate increases from $\sim$ 12% to $\sim$ 30% in the easiest cells), but the gap between the learned predictor and $K{=}1$ collapses to $<$ 3 cm in all cells (Fig. 5). Ablating `target_pos` alone, or both `target_pos` and `reach_err`, from the BC input does not restore differentiation: the BC policy's open-loop average reach trajectory does not actually consume per-step state estimates strongly enough to surface estimator differences. We treat this as a controller- side result and return to it in the discussion.

E. Long-horizon supervision flips the optimum (C3)

Retraining the forward model with K-step rollout supervision $\mathcal{L}_K$ drops $h{=}50$ tip prediction error to 0.064 m ($K{=}4$, -54% vs. baseline) and 0.048 m ($K{=}8$, -65%). The open-loop oracle then expands the $K^\star{<}1$ regime from 18 to 26 ($K{=}4$) and 27 ($K{=}8$) of 72 cells; blending becomes optimal at delay ≥ 6 steps in high-noise cells (Fig. 2, middle and right panels).

In *closed loop* the picture is opposite: under both $K{=}4$ and $K{=}8$, $K{=}0$ becomes the best estimator across all 6 cells of the focused grid, with min-tip dropping to $\sim$ 0.53 m ($K{=}4$) and $\sim$ 0.49 m ($K{=}8$) versus $K{=}1$ at 0.66–0.77 m. Notably $K{=}1$ *worsens* from 0.60 to 0.70 m as multi-step supervision strengthens at delay-18 cells: a stronger forward model makes the observation correction it is being asked to apply less compatible with the planned trajectory.

The condition-level learned predictor, however, keeps outputting $K{\approx}0.36$–0.99 (Fig. 6, panels 2–3) because the open-loop oracle it was trained on never picks $K{=}0$ — at delay ≥ 18 even with $K{=}8$ supervision, the open-loop oracle still selects $K^\star{=}1$ or close to it (the open-loop estimation error is minimised by trusting the observation when the buffer rollout is long). The optimum gap thus widens, not closes, with supervision strength.

F. Closed-loop-aware oracle resolves the gap (C4)

We address the gap by changing only the oracle definition. Under the $K{=}8$ forward model we sweep $K \in \{0, 0.25, 0.5, 0.75, 1.0\}$ in closed loop on the same 6 cells and pick $K^\star_{\text{cl}}$ per cell by minimising closed-loop min-tip error. The result is unanimous: $K^\star_{\text{cl}}{=}0$ in every cell. We retrain the same MLP g_ϕ on the 18 $K^\star_{\text{cl}}{=}0$ labels (3 controllers $\times$ 6 cells); the predictor now outputs $K{\approx}0.018$–0.025.

Plugged back into closed loop (Fig. 6, panel 4) the new predictor matches the $K{=}0$ baseline within 1.5 cm at every delay-18 cell (and slightly beats $K{=}0$ at (xhigh, $d{=}18$)). At delay-0 cells a $\sim$ 17 cm gap remains, because the closed-loop K-curve at $d{=}0$ jumps sharply from 0.49 m at $K{=}0$ to 0.77 m at $K{=}0.25$, and the sigmoid asymptote (0.025) loses $\sim$ 0.17 m. Across all 6 cells the closed-loop-aware predictor closes 40–60% of the single-step paradigm gap; mean improvement vs. the open-loop-oracle predictor is -16 cm. No architectural changes were made.

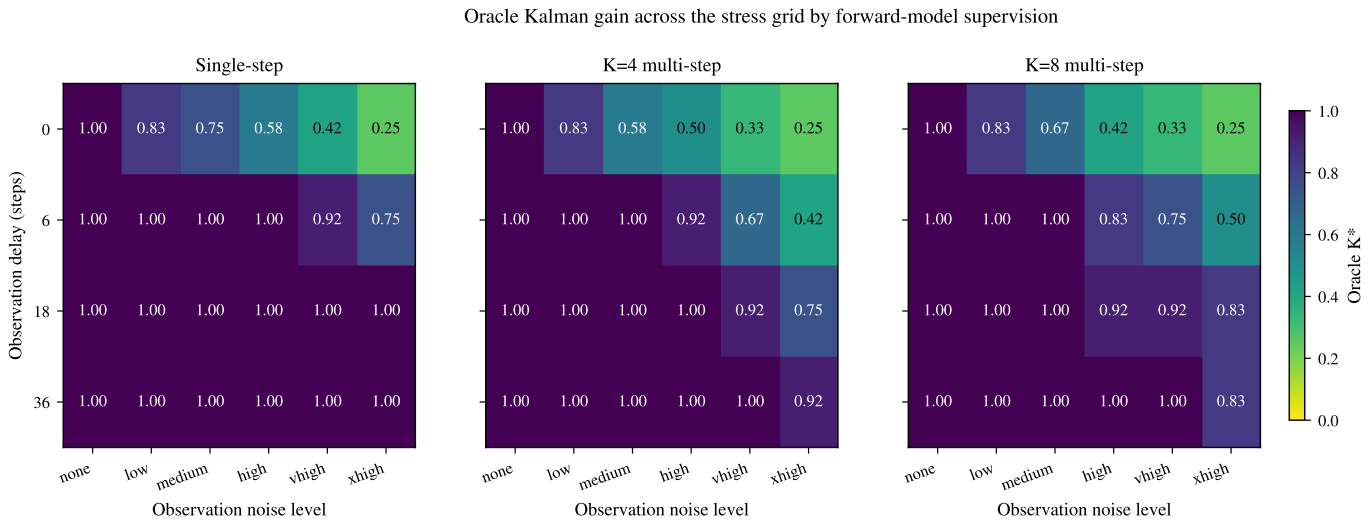

Fig. 2. **Open-loop oracle Kalman gain $K^\star$ as a function of observation delay (rows) and noise level (columns), shown for three forward-model variants.** Each cell reports the $K \in \{0, 0.25, 0.5, 0.75, 1.0\}$ that minimises the open-loop estimation error $\|\hat{x}_t - x_t\|^2$ on a stress-eval grid of 3 controllers $\times$ 6 noise levels $\times$ 4 delays $\times$ 5 K-values ($=$ 360 cells; the mean across the three controllers is shown). Dark purple marks $K^\star$=1 (trust the observation); yellow marks $K^\star$=0.25 (trust the model). Under single-step supervision (left) the $K^\star$<1 regime is confined to zero delay (18 / 72 cells); multi-step supervision widens it to delay $\geq$ 6 in the high-noise cells (26 / 72 for K=4, 27 / 72 for K=8).

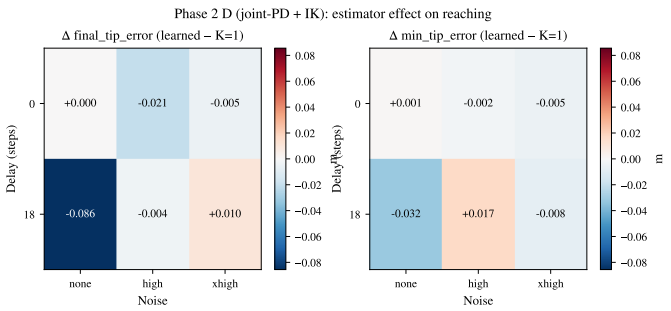

Fig. 3. **Closed-loop reaching: signed difference Δ = learned $-$ (K=1) in min-tip and final-tip error per (noise, delay) cell, under the single-step-supervised forward model and the joint-PD controller (mean over 10 episodes per cell).** Blue cells favour the learned predictor, red cells favour K=1, white cells are within $\pm$0.02 m. The $-$8.6 cm signal at (none, d=18) is the only cell where the learned predictor clearly beats K=1 — exactly the regime where the open-loop oracle of Fig. 2 says $K^\star$=1, but where in closed loop the controller's reliance on the buffered prediction makes a tighter estimate pay off. Other cells differ by less than 2 cm.

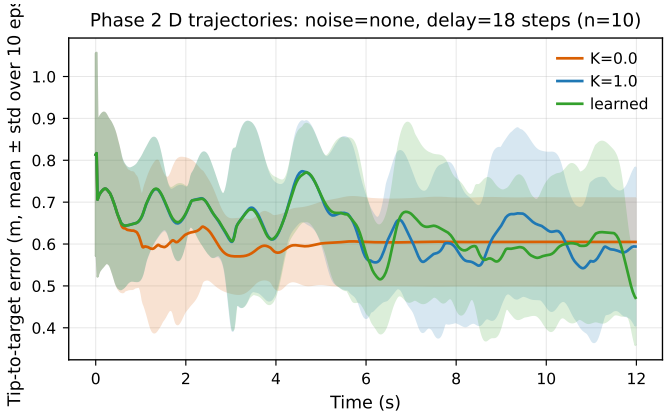

Fig. 4. **Closed-loop tip-to-target distance over a 12 s episode at (noise=none, d=18), under the single-step-supervised forward model and the joint-PD controller.** Lines and shaded bands are the mean $\pm$ standard deviation over 10 episodes for each estimator; the dashed line marks the 5 cm reach threshold. Prediction-only (K=0, orange) diverges over the d=18 buffer rollout; observation-only (K=1, blue) approaches but oscillates around the buffered target; the learned condition-level predictor (green) reaches further, producing the $-$8.6 cm advantage of Fig. 3.

V. Discussion

Forward-model accuracy reshapes the closed-loop optimum. The $K^\star$ that minimises open-loop estimation error is not the same as the $K^\star$ that minimises closed-loop task error (C3). As the forward model becomes accurate enough to roll out over the controller's planning horizon, observation correction starts to actively hurt rather than help: the condition-level predictor trained on the open-loop oracle is then mismatched by 0.5–0.7 in K from the closed-loop truth. We read this as a structural finding rather than an artifact: identical arm dynamics and observation noise produce different optimal estimators depending on whether the readout is the residual of a one-step update or the cumulative effect of feedback control over hundreds of steps.

Oracle redefine is a cheap fix. C4 shows that this mismatch is mostly resolvable by training the predictor on a closed-loop K-sweep rather than an open-loop one — no change to the predictor MLP or its capacity. The remaining $\delta \sim 17$ cm gap at zero delay is parameterisation-bound: a sigmoid cannot output K=0 exactly, and the closed-loop K-curve at d=0 is sharp around 0. A Hardtanh or output-clipped variant should close this further; we leave that to future work.

Behaviour cloning is not a good benchmark for state estimation. C2 shows that, at modest demonstration scale, a behaviour-cloned policy reaches better than the joint-PD controller but consumes state estimates so weakly that estimator differences disappear from task metrics. Closed-loop estimator benchmarks should use controllers that are demonstrably state-

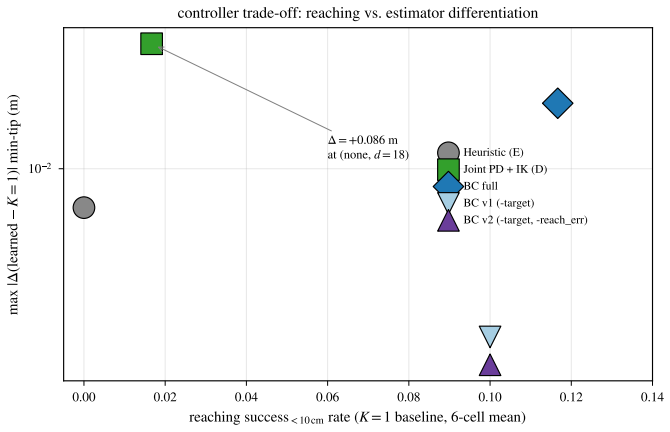

Fig. 5. **Each point is one controller variant evaluated on the same focused 6-cell grid: x-axis is the reaching success$_{<10\,\mathrm{cm}}$ rate under a $K{=}1$ baseline estimator, y-axis is the worst-case $|\Delta(\text{learned} - K{=}1)|$ in min-tip over the six cells.** The joint-PD+IK controller (D, green square) sits in the high-differentiation / low-reaching corner and is the only variant in which the learned predictor visibly beats $K{=}1$ (-8.6 cm at (none, $d{=}18$)); the heuristic controller (E) reaches nothing; the three BC variants (BC full, $-$target, $-$target/$-$reach_err) reach 8–12 % but their estimator differentiation collapses to under 3 mm. The y-axis uses symmetric log scaling.

coupled (the joint-PD+IK family used here works); a BC policy that generalises poorly per target is effectively open-loop in the state channel even though it nominally consumes a state vector.

Limits and open questions. (a) The closed-loop oracle sweep we use is expensive ($K_{\mathrm{grid}}\times$ cells $\times$ episodes rollouts); a differentiable surrogate would scale this far better. (b) Our learned predictor is condition-level (no per-step state coupling beyond the sigma vector and delay scalar); a state-aware Stage B variant we explored briefly did not beat the condition-level baseline because of labelling noise, but a closed-loop differentiable label might. (c) All results are on a single MyoSuite reach task; transfer to grasp/place tasks is open. (d) The 17 cm residual at $d{=}0$ is structural and could be closed with a predictor parameterisation that supports $K{=}0$ exactly.

VI. Conclusion

We have shown on a muscle-driven reaching task that the closed-loop optimal Kalman gain depends on both the forward model's accuracy and the oracle used to train any learned gain predictor. Single-step supervision of the forward model creates a small but real regime where a learned condition-level gain beats observation-only by 8.6 cm at the delay-dominated cell (C2). Multi-step supervision flips the regime: $K{=}0$ becomes globally optimal in closed loop, with the gap to $K{=}1$ growing to 12–24 cm (C3). Redefining the predictor's training oracle in terms of closed-loop task error rather than open-loop estimation accuracy aligns the predictor with the new regime and recovers most of the gap (C4) with no architectural changes. We argue that learned- Kalman-gain papers should specify the oracle target explicitly and that closed-loop benchmarks should be a standard part of evaluating adaptive state estimators.

Acknowledgement

The author thanks the MyoSuite maintainers for releasing the musculoskeletal benchmark used in this study.

References

[1] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Transactions of the ASME–Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, Mar. 1960.

[2] R. K. Mehra, "On the identification of variances and adaptive Kalman filtering," *IEEE Transactions on Automatic Control*, vol. 15, no. 2, pp. 175–184, Apr. 1970.

[3] B. D. O. Anderson and J. B. Moore, *Optimal Filtering*. Englewood Cliffs, NJ: Prentice-Hall, 1979, republished by Dover, 2005.

[4] N. A. Bernstein, *The Coordination and Regulation of Movements*. Oxford: Pergamon Press, 1967.

[5] D. M. Wolpert and Z. Ghahramani, "Computational principles of movement neuroscience," *Nature Neuroscience*, vol. 3, no. 11, pp. 1212–1217, Nov. 2000.

[6] V. Caggiano, H. Wang, G. Durandau, M. Sartori, and V. Kumar, "MyoSuite – a contact-rich simulation suite for musculoskeletal motor control," in *Proc. 4th Annual Learning for Dynamics and Control Conference (L4DC)*, ser. Proceedings of Machine Learning Research, vol. 168. PMLR, 2022. [Online]. Available: https://proceedings.mlr.press/v168/caggiano22a.html

[7] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.

[8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[9] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2020. [Online]. Available: https://arxiv.org/abs/1912.01603

[10] M. Janner, J. Fu, M. Zhang, and S. Levine, "When to trust your model: Model-based policy optimization," in *Advances in Neural Information Processing Systems*, vol. 32, 2019. [Online]. Available: https://proceedings.neurips.cc/paper/2019/hash/5faf461eff3099671ad63c6f3f094f7f-Abstract.html

[11] H. E. Rauch, F. Tung, and C. T. Striebel, "Maximum likelihood estimates of linear dynamic systems," *AIAA Journal*, vol. 3, no. 8, pp. 1445–1450, Aug. 1965.

[12] D. A. Pomerleau, "ALVINN: An autonomous land vehicle in a neural network," in *Advances in Neural Information Processing Systems*, vol. 1. Morgan Kaufmann, 1988. [Online]. Available: https://proceedings.neurips.cc/paper/1988/hash/812b4ba287f5ee0bc9d43bbf5bbe87fb-Abstract.html

[13] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. 14th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 15. PMLR, 2011, pp. 627–635. [Online]. Available: https://proceedings.mlr.press/v15/ross11a.html

Appendix A
Hyperparameters and Implementation Details

Table I lists every hyperparameter used in the paper. All training runs are reproducible from `configs/` in the source repository; estimator evaluation grids are listed verbatim in `configs/estimators/` and `configs/closed_loop/`.

Appendix B
Forward-Model Rollout Metrics

Table II reports horizon-wise prediction error for the three forward-model variants used in the paper. Multi-step rollout supervision substantially reduces medium- and long-horizon error while paying a small cost at $h{=}1$.

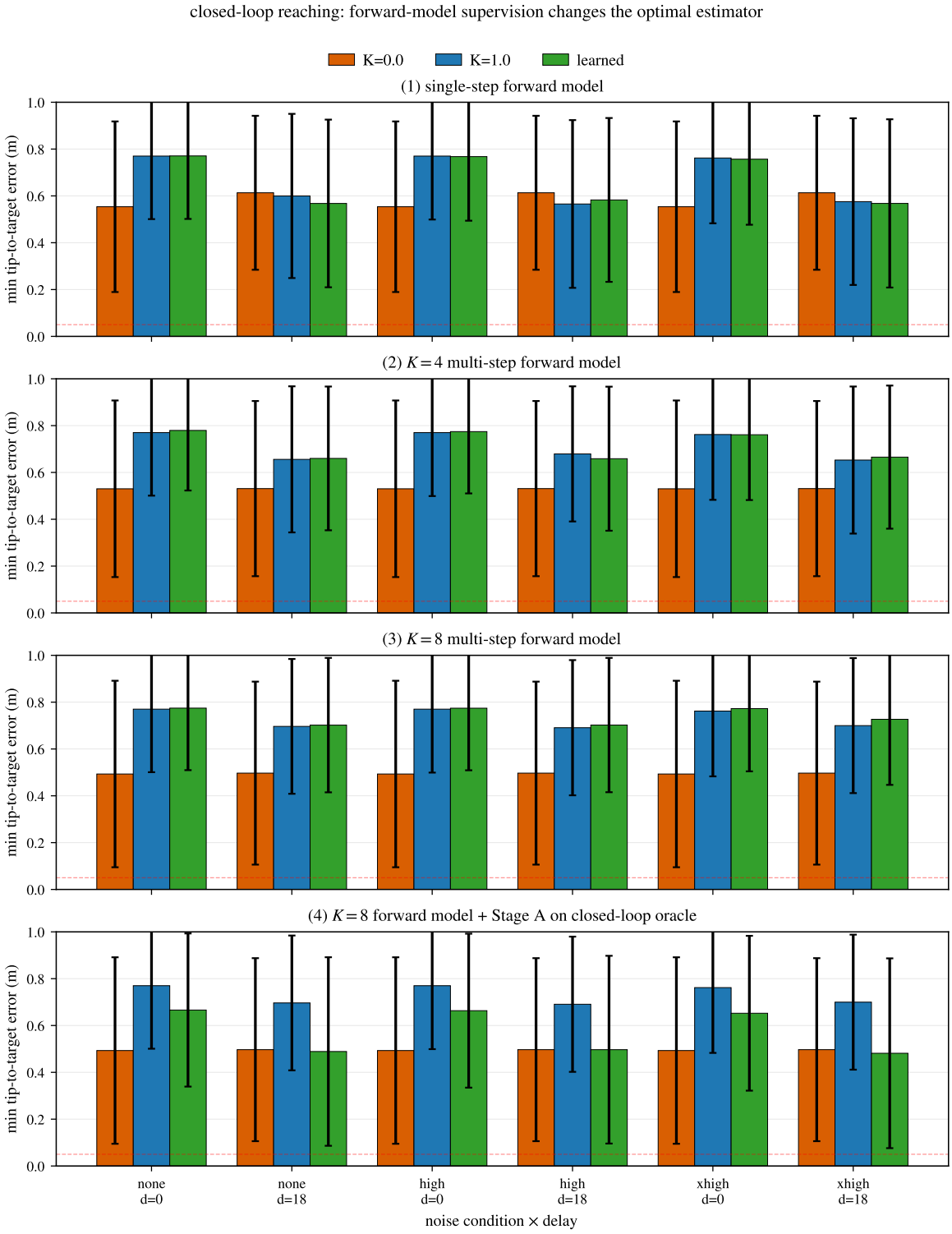

Fig. 6. **Closed-loop min-tip reaching error under the joint-PD controller across four (forward-model, oracle) variants.** Each panel reports mean ± standard deviation over 10 episodes per cell for three estimators: $K{=}0$ (prediction-only, orange), $K{=}1$ (observation-only, blue), and the learned condition-level predictor g_ϕ (green). The dashed horizontal line marks the 5 cm reach threshold. (1) Single-step-supervised forward model: $K{=}0$ diverges in some cells, the learned predictor sits near $K{=}1$. (2) $K{=}4$ multi-step supervision: $K{=}0$ becomes competitive. (3) $K{=}8$ multi-step: $K{=}0$ is best in every cell while the learned predictor still tracks $K{=}1$ because its open-loop oracle never selected $K{=}0$. (4) Same $K{=}8$ forward model, but g_ϕ retrained against the closed-loop oracle (Sec. IV-F): the green learned bar now matches the orange $K{=}0$ bar at the three delay-18 cells, closing 40–60 % of the gap at the delay-0 cells.

TABLE I
HYPERPARAMETERS USED ACROSS THE PAPER.

Component	Hyperparameter	Value
Forward model f_θ	state dim	83
	action dim	34
	hidden layers	2×256
	activation	ReLU, LayerNorm
	output	residual (Δx)
	parameters	118,355
Forward training	optimiser	Adam [8]
	learning rate	1×10^{-3}
	weight decay	0
	batch size	256
	epochs	200 (early stop patience 20)
	val split	every 5th source episode
Multi-step loss	K values tested	1, 4, 8
Stage A g_ϕ	input dim	8 = 3 ctrl + 4 σ + 1 delay/$d_{\max}$
	hidden layers	2×32
	output	sigmoid scalar in $[0, 1]$
	parameters	1,409
Stage A training	optimiser	Adam
	learning rate	1×10^{-3}
	weight decay	1×10^{-3}
	epochs	500
Joint-PD controller	K_p, K_d	30, 3
	action scale α	5
	muscle map	$\mathrm{clip}(\alpha\,\mathrm{ReLU}(M\,u_{\mathrm{joint}}))$, $M \in \mathbb{R}^{34\times 20}$
IK solver	method	damped Jacobian pseudoinverse
	max iter / tol	200 / 0.01 m
	damping λ^2	0.1
BC policy	state dim variants	83 (full) / 80 / 77
	hidden layers	2×256
	output	sigmoid 34-dim
	demos / training	30 ep. $\times$ 600 steps; 100 epochs
Eval grid (open-loop)	noise levels	none, low, medium, high, vhigh, xhigh
	delays (steps)	0, 6, 18, 36
	K values	0.0, 0.25, 0.5, 0.75, 1.0
Eval grid (closed-loop)	noise levels (focused)	none, high, xhigh
	delays (focused)	0, 18

TABLE II
FORWARD-MODEL ROLLOUT MSE AND TIP PREDICTION ERROR ON THE HELD-OUT VALIDATION EPISODES. LOWER IS BETTER. ALL THREE MODELS SHARE THE SAME ARCHITECTURE AND TRAINING SET (51,534 TRANSITIONS ACROSS 106 EPISODES); THEY DIFFER ONLY IN THE MULTI-STEP VALUE K.

	MSE			Tip err. (m)		
Model	h=1	h=10	h=50	h=1	h=10	h=50
K=1 (single-step)	0.0055	0.0124	0.0524	0.0076	0.0355	0.1381
K=4 multi-step	0.0053	0.0084	0.0164	0.0088	0.0247	0.0635
K=8 multi-step	0.0055	0.0082	0.0114	0.0098	0.0215	0.0483

APPENDIX C
FULL OPEN-LOOP ORACLE $K^\star$ GRIDS

Table III reports the mean oracle $K^\star$ per (delay, noise) cell under the K=8 multi-step-supervised forward model (used by Sec. IV-E/3.6). The K=4 and single-step matrices appear in Fig. 2 of the main text.

TABLE III
ORACLE KALMAN GAIN $K^\star$ AS A FUNCTION OF DELAY AND NOISE UNDER THE K=8 MULTI-STEP-SUPERVISED FORWARD MODEL (MEAN OVER 3 CONTROLLERS; 72 CONDITIONS TOTAL). 27 OF 72 CELLS REQUIRE $K^\star$<1.

	noise					
delay	none	low	medium	high	vhigh	xhigh
0	1.000	0.833	0.667	0.417	0.333	0.250
6	1.000	1.000	1.000	0.833	0.750	0.500
18	1.000	1.000	1.000	0.917	0.917	0.833
36	1.000	1.000	1.000	1.000	1.000	0.833

APPENDIX D
CLOSED-LOOP K-SWEEP BACKING C4

Table IV reports the closed-loop min-tip-to- target error per cell as a function of the (fixed) Kalman gain K, under the K=8 forward model and the joint-PD controller (10 episodes per cell). The K-sweep is the basis of the closed-loop oracle $K^\star_{\mathrm{cl}}$ defined in Sec. IV-F. $K^\star_{\mathrm{cl}}$=0 in every cell.

Two regularities are worth noting. First, at zero delay the K-curve is sharp around K=0: moving from K=0 to K=0.25 jumps min-tip from 0.49 m to 0.77 m. The Stage A predictor's sigmoid asymptote ($K \approx 0.025$) sits on the wrong side of this jump in delay-0 cells, which is why the residual gap remains ($\sim$ 17 cm; see Sec. IV-F). Second, at delay-18 the K-curve is smooth near K=0 (K=0 and K=0.25 within 0.01–0.05 m) so the predictor's small offset is harmless and the same predictor matches the K=0 baseline within 1.5 cm.

TABLE IV
CLOSED-LOOP min-TIP ERROR (M) PER CELL ACROSS THE K-SWEEP UNDER THE K=8 FORWARD MODEL AND THE JOINT-PD CONTROLLER. EACH CELL AVERAGES 10 EPISODES. BOLD MARKS THE PER-CELL MINIMUM.

noise	delay	K=0	K=0.25	K=0.5	K=0.75	K=1
none	0	**0.493**	0.771	0.782	0.774	0.770
none	18	**0.497**	0.498	0.708	0.746	0.697
high	0	**0.493**	0.767	0.780	0.771	0.770
high	18	**0.497**	0.538	0.696	0.746	0.691
xhigh	0	**0.493**	0.772	0.769	0.766	0.762
xhigh	18	**0.497**	0.550	0.713	0.749	0.700

APPENDIX E
SOFTWARE, HARDWARE, AND REPRODUCIBILITY

All experiments run on a single workstation (no GPU is required; the forward model and Stage A predictor are small enough that CPU training takes only minutes). Software stack: Python 3.11, PyTorch (CPU), MuJoCo 2.3, MyoSuite 2.12.2, Gymnasium 0.29. Reproduction scripts and exact dataset paths are recorded in the source repository's `docs/02_InitialImplementationPlan.md` and `docs/03_PaperOutline.md`.